

scheduled-work-engine

Revision: 1 Last modified: 2026-07-02T00:00:00Z

A decoupled, project-agnostic **scheduled-work / reminder tracking service**. It records background-queue work items and lets an autonomous agent — when a REMINDER fires — re-verify **uncertain / blocked / overdue** work before reporting it done (constitution §11.4.6 no-guessing, §11.4.108 verify-before-report). It is the backing infrastructure for the REMINDER / BACKGROUND action-prefix (§11.4.140).

Surfaces (one implementation, multiple transports)

Surface	Transport	Entry
REST	HTTP/3 (QUIC) + h2/h1 fallback, brotli	<code>serve</code> → <code>/api/v1/...</code>
MCP	HTTP/3 + h2/h1 (JSON-RPC 2.0)	<code>serve</code> → <code>POST /mcp</code>
MCP	stdio (Claude Code plugin)	<code>mcp-stdio</code>
ACP-equivalent	stdio (JSON-RPC 2.0)	<code>acp-stdio</code>

Router = `gin` (`gin.Engine` is served identically over h1/h2/h3); HTTP/3 = `github.com/quic-go/quic-go/http3` ; compression = `github.com/andybalholm/brotli` middleware.

Build

```
go build -o bin/scheduled-work ./cmd/scheduled-work # or: bash ../../plugins/scheduled-work/build.sh
```

The binary is a build derivative (gitignored per §11.4.30; regen §11.4.77).

Run

```
# REST + MCP-over-HTTP on HTTP/3 (+ h2/h1) with a self-signed dev cert:
```

```
SWQ_ADDR=127.0.0.1:8787 ./bin/scheduled-work serve
```

```
# MCP over stdio (for a Claude Code plugin / any MCP stdio client):
```

```
./bin/scheduled-work mcp-stdio
```

```
# ACP-equivalent JSON-RPC over stdio:
```

```
1 1 4 2 8
```

```
./bin/scheduled-work acp-stdio
```

Configuration (env – \$. . decoupling:
no hardcoded consumer data)

Var	Default	Meaning
SWQ_ADDR	127.0.0.1:8787	listen address (TCP for h1/h2, UDP for h3)
SWQ_DB	./scheduled_work.json	JSON store path
SWQ_TLS_CERT / SWQ_TLS_KEY	(self-signed dev)	production TLS pair
SWQ_BROTLI	5	brotili quality 0–11, or -1 to disable
SWQ_H3	1	1 enable HTTP/3, 0 = h1/h2 only

REST API

Method	Path	Purpose
GET	/healthz	liveness + item count
POST	/api/v1/items	create {title, description?, status?, due_at?, tags?, source?, notes?}
GET	/api/v1/items?status=	list (effective-status filter, incl. overdue)
GET	/api/v1/items/over-due	open items past due
GET	/api/v1/items/needs-verification	open blocked / uncertain / overdue
GET	/api/v1/items/:id	one item
PATCH	/api/v1/items/:id/status	{status, notes?}
POST	/api/v1/items/:id/done	{notes?} — mark done (put evidence in notes)
DELETE	/api/v1/items/:id	delete

MCP tools

create_work_item , list_work_items , get_work_item ,
update_work_item_status , mark_work_item_done , list_overdue_work ,
list_needs_verification . See skills/scheduled-work-queue/skill.md .

Status model (closed set)

`scheduled` · `in-progress` · `blocked` · `uncertain` · `done` · `cancelled` ,
plus a **derived** `overdue` (an open item past `due_at` , computed on read, never
stored — §11.4.6). `blocked` / `uncertain` / `overdue` are the states a
REMINDER MUST confirm before reporting done.

Testing

```
go test ./...           # unit (store, mcp)
go test -race ./test/   # integration: REAL HTTP/3 + brotli round-trip
bash test/meta_mutation.sh    # §1.1 paired mutation (RED-on-broken / GREEN-on-restore)
bash test/live_smoke.sh      # live captured-evidence smoke (REST TLS + MCP stdio)
```

See `DESIGN.md` for the store-choice justification, the ACP honesty note, and the
proposed governance-anchor handoff.